RESEARCH-ARTICLE

SAPO: Improving the Scalability and Accuracy of Quantum Linear Solver for Portfolio Optimization

TIANZE ZHU, Zhejiang University, Hangzhou, Zhejiang, China

LIQIANG LU, Zhejiang University, Hangzhou, Zhejiang, China

JIAJUN CHEN, Zhejiang University, Hangzhou, Zhejiang, China

YUHANG CHEN, Peking University, Beijing, China

HENGRUI CHEN, Zhejiang University, Hangzhou, Zhejiang, China

MENG XI, Zhejiang University, Hangzhou, Zhejiang, China

View all

Published: 22 June 2023

Citation in BibTeX format

DAC '25: 62nd Annual ACM/IEEE Design Automation Conference
June 22 - 26, 2023
California, San Francisco, United States

Conference Sponsors:
SIGDA

SAPO: Improving the Scalability and Accuracy of Quantum Linear Solver for Portfolio Optimization

Tianze Zhu
Zhejiang University
Hangzhou, China
tianzezhu@zju.edu.cn

Liqiang Lu*
Zhejiang University
Hangzhou, China
liqianglu@zju.edu.cn

Jiajun Chen
Zhejiang University
Hangzhou, China
cuiox@zju.edu.cn

Yuhang Chen
Peking University
Beijing, China
chenyuhang@pku.edu.cn

Hengrui Chen
Zhejiang University
Hangzhou, China
hengruichen@zju.edu.cn

Meng Xi
Zhejiang University
Hangzhou, China
ximeng@zju.edu.cn

Jinshan Zhang
Zhejiang University
Hangzhou, China
zhangjinshan@zju.edu.cn

Xiaoming Sun
Chinese Academy of Sciences
Beijing, China
sunxiaoming@ict.ac.cn

Jianwei Yin
Zhejiang University
Hangzhou, China
zjuyjw@cs.zju.edu.cn

2025 62nd ACM/IEEE Design Automation Conference (DAC) | 979-8-3315-0304-8/25/$31.00 ©2025 IEEE | DOI: 10.1109/DAC63849.2025.11133130

***Abstract*—Portfolio optimization is one of the most important financial problem, suffering from huge computational pressure due to arithmetic complexity. Quantum computing offers polynomial or even exponential speedup that turns out to be a promising approach. However,existing quantum methods is fundamentally limited by either poor scalability or insufficient accuracy. In this paper, we propose SAPO, which formally articulates the quantum circuit that seamlessly integrates financial theory and historical data characteristics with quantum algebra. The circuit design is extended from the HHL algorithm incorporating mean-variance theory, which promotes scalability by equivalent transformation. Then, we present a min-max eigenvalue model that leverages historical financial information to refine parameter settings with high accuracy. Experiments conducted on market data demonstrate that SAPO can effectively reduce the complexity by 36.94% compared to basic HHL [1], [2] and improve the accuracy by 1.46× compared to hybrid HHL [3].**

I. Introduction

Portfolio optimization identifies the best asset allocations to balance the risk and return, which is grappling with computational bottlenecks [4]. Through qubit entanglment and superposition, quantum computing offers an alternative for portfolio optimization. Extending from the mean-variance portfolio theory [5], classical solutions introduce various models, including the standard model, Tobin-Sharpe-Lintner model, and Black's model [6]. It has been demonstrated in recent researches that quantum algorithms are particularly well-suited for optimizing portfolios subject to the simple equality constraints in Black's model [7], [8].

Quantum approaches can be categorized into the quantum Ising solver [9], [10], [11] and the Harrow-Hassidim-Lloyd (HHL) algorithm [1], [2], [3]. The quantum Ising solver handles a quadratic unconstrained binary optimization problem, where asset holdings are represented by superpositions of 0 or 1, with the final strategy determined by the qubit state. On the other hand, the HHL algorithm formulates Black's model as a linear system, computing allocations by projecting the vector space onto quantum states. Despite the significant parallelism of quantum computing, existing attempts listed in Table I suffer from either poor scalability or low accuracy. The quantum Ising solver only tell the strategy of "buy" or "not buy". In other words, it lacks the capability to quantify the weight for each asset, leading to low accuracy. Conversely, the HHL algorithm can provide concrete weights. However, it exhibits overwhelming computational complexity in terms of circuit scale. For example, the circuit for 2-assets portfolio optimization has a depth over 3.7×10^8 after being compiled by Qiskit [12]. Though there are hybrid HHL algorithms [3] attempt to reduce the circuit complexity by inserting classical operations, they still rely on numerical properties and require massive qubits to obtain precise eigenvalues.

*Corresponding author.

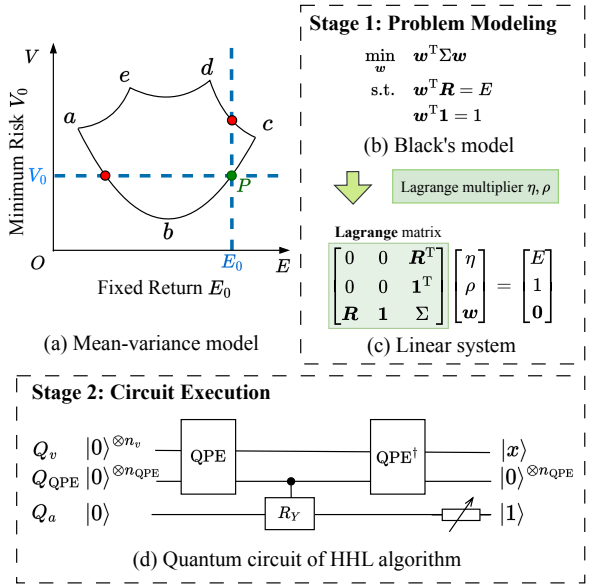


Fig. 1. Portfolio optimization using HHL algorithm.

These limitations originate from the absence of financial theory and historical financial information. When applied to portfolio optimization, the quantum Ising solver is hard to involve the intrinsic features. Consequently, the implementation on a quantum annealer [9] or parameterized quantum circuit [10], [11] is a coarse-grained coding scheme, resulting in overly simplistic recommendations for investment. On the other hand, the drawbacks of existing HHL algorithms are primarily attributed to the complexity of solving the linear system, fundamentally bounded by the large condition number of the matrix in it. Specifically, the condition number serves as an indicator when solving linear equations [13], which is a squared term in the asymptotic complexity of the HHL algorithm [1]. For 2-asset portfolio optimization, the condition number can reach an order of 10^5. Even with the hybrid method [3], which reduces the number of controlled rotation gates by inserting measurements in the midst of the HHL algorithm, it fails to mitigate the substantial complexity of the HHL circuit. Moreover, its accuracy hinges on the matrix's eigenvalue characteristics, making it difficult to estimate the necessary qubit number for the quantum phase estimation (QPE) module.

To overcome these challenges, we propose SAPO that exploits financial theory and data to improve both scalability and accuracy for portfolio optimization. Building upon the HHL algorithm, we observe that previous approaches are significantly limited by the large condition number, leading to poor scalability. To tackle this, we propose a novel constraint scaling technique with equivalent transformation to

TABLE I
EXISTING QUANTUM APPROACHES FOR PORTFOLIO OPTIMIZATION.

Methodology	**quantum Ising solver**			**HHL algorithm**		
	HyQSAT [9]	FrozenQubits [11]	CAFQA [10]	basic HHL [1], [2]	hybrid HHL [3]	**SAPO**
Parameter setting	✘	✘	Clliford simulation	✘	✘	eigenvalue predictor
Resource saving	subset identification	freeze hotspot nodes	✘	✘	middle measurement	condition number reduction
Financial theory	✘	✘	✘	✘	✘	Black's model & historical info.
Complexity level	low	low	low	high	medium	medium
Accuracy level	only tell "buy" or "not buy"			medium	medium	high

shrink it while adhering to the principles of Black's model. Then we develop an iterative algorithm to identify the optimal scaling factors for a given financial dataset. Moreover, with the economic understanding, we present a meticulous circuit parameterization to further improve the accuracy. This is grounded in prior knowledge derived from historical financial data, making it a static model that only depends on the dataset.

The contribution of this work is summarized as follows:

- This paper represents an exploratory endeavor aimed at an in-depth understanding of applying quantum computing to financial scenarios. The key is to profoundly leverage the intricacies of financial knowledge and data. We investigate the optimization opportunities of Black's model at both arithmetic level and quantum circuit level.
- We propose the constraint scaling technique for significant reduction of the condition number, which exploits the equality constraints in Black's model, leading to low complexity.
- We propose a prediction model, which estimates the minimum and maximum eigenvalues to delicately parameterize the quantum circuit, resulting in high accuracy.

In our experiments, SAPO achieves around 34.64%-36.94% complexity reduction compared to basic HHL [1], [2] with different settings of the absolute error. Furthermore, compared to Qiskit HHL [12] and hybrid HHL [3], SAPO improves the accuracy by $1.52\times$ and $1.46\times$ under fixed qubit number, and by $1.28\times$ and $1.39\times$ under fixed asset number, respectively.

II. BACKGROUND

A. Portfolio Optimization

H. Markowitz details the *mean-variance theory* for portfolio optimization [5], which aims to find the optimal asset allocation vector $\boldsymbol{w}$, with the objective of minimizing the risk while maximizing the expected return. First, we consider the returns of n assets across each of the T time periods, denoted as:

$$\boldsymbol{r}_t = \left[r_1^t, r_2^t, \cdots, r_n^t\right]^{\mathrm{T}}, \quad t = 1, 2, \cdots, n \tag{1}$$

Subsequently, the historical expected return vector, denoted as $\boldsymbol{R}$, and the covariance matrix, denoted as Σ, can be estimated using the data from the past T time slices as follows:

$$\begin{aligned} \boldsymbol{R} &\equiv \mathbb{E}(\boldsymbol{r}_t) \approx \frac{1}{T}\sum_{t=1}^{T} \boldsymbol{r}_t \\ \Sigma &\equiv \mathbb{V}(\boldsymbol{r}_t) \approx \frac{1}{T-1}\sum_{t=1}^{T} (\boldsymbol{r}_t - \boldsymbol{R})^{\mathrm{T}} (\boldsymbol{r}_t - \boldsymbol{R}) \end{aligned} \tag{2}$$

Under a given $\boldsymbol{w}$, the expected return E and risk V can be defined:

$$\begin{aligned} E &\equiv \boldsymbol{w}^{\mathrm{T}} \boldsymbol{R} \\ V &\equiv \boldsymbol{w}^{\mathrm{T}} \Sigma \boldsymbol{w} \end{aligned} \tag{3}$$

For instance, as depicted in Fig. 1(a), the optimal $\boldsymbol{w}$ corresponds to the minimized risk V_0 for a specified expected return E_0. Typically, the problem can be formulated as a quadratic programming problem, where $\boldsymbol{w}$ constitutes the solution (Fig. 1(b)). In particular, Black's model [6] presents the problem with equality constraints. By employing the Lagrange multiplier method, the optimization can be converted into a linear system with a Lagrange matrix M_L (Fig. 1(c)).

B. HHL Algorithm

Considering A as a Hamiltonian matrix with eigenvalues $\{\lambda_j\}_{j=1}^n$ and eigenvector $\{|\mu\rangle_j\}_{j=1}^n$ and $\boldsymbol{b}$ as a normalized vector represented by $\sum_{j=1}^n \beta_j |\mu_j\rangle$. The solution to the linear system $A\boldsymbol{x} = \boldsymbol{b}$ can be represented as a quantum state:

$$|x\rangle = \sum_{j=1}^{n} \frac{\beta_j}{\lambda_j} |\mu_j\rangle \tag{4}$$

Fig. 1(d) illustrates the quantum circuit of the HHL algorithm [1]. The circuit is comprised of three primary components, including QPE, uniformly controlled R_Y gate, and QPE†. There are also three quantum registers for different functions: a) Q_v is used to load the vector $\boldsymbol{b}$ as part of the QPE module; b) Q_{QPE} stores the output of QPE, recording eigenvalue information; c) Q_a only contains an auxiliary qubit controlled by Q_{QPE} through the uniformly controlled R_Y gate.

- **QPE module.** In the HHL algorithm, the QPE part is used to extract eigenvalues of the input Hamiltonian matrix A. Specifically, QPE generates a superposition state of phases $\varphi \in [0, 1)$ and records this in the quantum register Q_{QPE}, establishing an entanglement with Q_v. Each phase and eigenvalue adhere to the relationship:
$$e^{2\pi i \varphi_j} = e^{i\lambda_j H_t} \tag{5}$$
where H_t is a key hyperparameter named *Hamiltonian simulation time*. Increasing the number of qubits for the QPE module can improve the preciseness of the phase, making the eigenvalue obtained by Eq. (5) more accurate. Thus, the number of qubits in Q_{QPE},represented as n_{QPE}, is an important hyperparameter. In addition, QPE† is the inverse operation of QPE, which is applied at the end of the quantum circuit to disentangle Q_v and Q_{QPE}.
- **Uniformly Controlled R_Y Gate.** The uniformly controlled R_Y gate is a multi-qubit controlled gate. In the HHL algorithm, according to the phase φ_j stored in Q_{QPE}, $R_Y(2\arcsin(Const/\lambda_j))$ gate is applied to the single qubit stored in Q_a so that the eigenvalue can be placed on the denominator of the amplitude of the quantum state. Here, $Const$ is a *normalization constant*, another critical hyperparameter in the HHL algorithm.

The result is obtained by measuring the auxiliary qubit stored in Q_a at the end of the quantum circuit. Ideally, if the result is 1, the quantum state collapses to the normalization of Eq. (4). For the sake of simplicity, this paper assumes that we can directly obtain

the quantum state vector after executing the circuit, which is post-processed to get the desired solution, denoted as $\tilde{\boldsymbol{w}}$.

III. SAPO Overview

A. Evaluation Metrics

As aforementioned, our goal is to improve the scalability and accuracy in portfolio optimization. To achieve this, we define two metrics, named *complexity* and *accuracy*, to evaluate quantum approaches.

First, we model the complexity in both spatial and temporal dimensions by counting the consumed quantum resources and the circuit depth. According to the HHL algorithm, the total number of qubits N^q is the summation of three quantum registers.

$$N^q = n_v + n_{\text{QPE}} + n_a \tag{6}$$

Besides, we use N^{CNOT} and N^d to represent the number of CNOT gates and the circuit depth after compilation. This is because the CNOT gate is the dominant source of errors on most existing NISQ devices [14], and the circuit depth is also an important indicator to evaluate the complexity of quantum algorithm [15]. Putting it all together, we formulate the complexity metric as follows.

$$\Theta = N^q + \lg\left(N^{\text{CNOT}}\right) + \lg\left(N^d\right) \tag{7}$$

Here, for simplicity, we employ lg function on N^{CNOT} and N^{d} since these two terms exhibit exponential growth with the number of qubits.

The accuracy is evaluated by quantifying the discrepancy between the calculated allocation $\tilde{\boldsymbol{w}}$ and the optimal choice $\boldsymbol{w}$. Thus, we formulate the accuracy in terms of cosine similarity:

$$\Delta = \frac{1}{2}\left(1 + \frac{\boldsymbol{w}^{\text{T}}\tilde{\boldsymbol{w}}}{\|\boldsymbol{w}\|\|\tilde{\boldsymbol{w}}\|}\right) \tag{8}$$

where Δ always falls within the interval $[0, 1]$. The closer it is to 1, the more accurate it is.

B. SAPO Advantages

The novelty of SAPO is that it leverages the knowledge of financial theory and data to optimize the design of the quantum approach. Extending from the Black's model, we observe that the condition number of the Lagrange matrix derived from real market data is typically large. This could markedly increase the circuit complexity, thereby limiting the scalability with respect to problem size. To address this, we propose a constraint scaling technique that adopts an equivalent transformation.

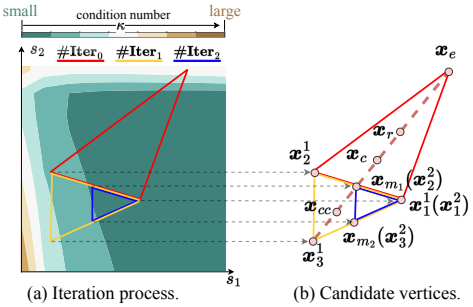
Next, we propose min-max eigenvalue model to enhance the accuracy of portfolio optimization. Our approach relies on the insight that knowing only the minimum and maximum eigenvalues of the Lagrange matrix is sufficient for setting the hyperparameters of the HHL circuit. We provide a quantitative formulation linking circuit hyperparameters to these eigenvalues obtained from a static predictor trained on historical data. In a word, we demonstrate that SAPO exhibits a lower end-to-end complexity compared to previous methods while achieving much higher accuracy for portfolio optimization.

IV. Equivalent Scaling of Lagrange Matrix

A. Equivalent Transformation

For the Lagrange matrix M_L featuring a set of eigenvalues $\{\lambda_j\}$, we first define min/max eigenvalue $|\lambda|_{\min} = \min\{|\lambda||\lambda \in \{\lambda_j\}\}$ and $|\lambda|_{\max} = \max\{|\lambda||\lambda \in \{\lambda_j\}\}$. We start by optimizing the condition number $\kappa = |\lambda|_{\max}/|\lambda|_{\min}$ of the Lagrange matrix.

Mathematically, the condition number indicates the difficulty of matrix calculus using approximation algorithms [13]. For example, when the condition number is exactly one, it means that the matrix is a scalar multiple of a linear isometry. In contrast, a larger condition number suggests that the linear system is hard to solve using an approximation solution, e.g., Jacobi method [16] and conjugate gradient method [17]. According to the HHL algorithm [1], the upper-bound complexity is $O(\log n \cdot \kappa^2/\epsilon)$, where n is the matrix size and ϵ is the given upper bound of absolute error. In the portfolio optimization problem, the condition number is usually in the scale of 10^5, which implies a huge complexity.



(a) Iteration process. (b) Candidate vertices.

Fig. 2. iterative method for searching the optimal scaling factor.

Conventional method to deal with large condition number involves rescaling the matrix by rows or columns [18]. However, such rescaling fails to keep the equivalence, reducing the expressiveness of the linear system. Thus, this method has to introduce various parameters that need to be tuned to compensate for the gap. Instead of directly scaling the matrix, we choose to modify Black's model by multiplying two scaling factors s_1, s_2 with the equations in Fig. 1(b).

$$\begin{aligned} \min_{\boldsymbol{w}} \quad & \boldsymbol{w}^{\text{T}}\Sigma\boldsymbol{w} \\ \text{s.t.} \quad & s_1\boldsymbol{w}^{\text{T}}\boldsymbol{R} = s_1 E \\ & s_2\boldsymbol{w}^{\text{T}}\boldsymbol{1} = s_2 \end{aligned} \tag{9}$$

In this manner, we have an equivalent Lagrange matrix as follows, which is an equivalent transformation with only two scaling factors.

$$\begin{bmatrix} 0 & 0 & s_1\boldsymbol{R}^{\text{T}} \\ 0 & 0 & s_2\boldsymbol{1}^{\text{T}} \\ s_1\boldsymbol{R} & s_2\boldsymbol{1} & \Sigma \end{bmatrix} \begin{bmatrix} \eta \\ \rho \\ \boldsymbol{w} \end{bmatrix} = \begin{bmatrix} s_1 E \\ s_2 \\ \boldsymbol{0} \end{bmatrix} \tag{10}$$

B. Search Algorithm of Scaling Factor

The scaling factor helps to adjust the condition number, providing the opportunity to reduce the difficulty of portfolio optimization. Empirically, we test that outstanding scaling factors can significantly reduce the condition number from 10^5 to ˜10. However, the optimal scaling factor varies in different Lagrange matrices. Therefore, we try to find a static scaling factor on a target dataset, which achieves optimal reduction on average in this dataset.

We adopt a iterative algorithm inspired by Nelder-Mead simplex algorithm [19] to search the optimal scaling factor. Here, we use a two-dimensional vector $\boldsymbol{x}$ to notate the tuple of (s_1, s_2) as shown in Fig. 2(a). First, given a set of Lagrange matrices derived from a specific dataset, we define a loss function that calculates the average of condition numbers after applying the scaling factor. The algorithm begins with the initialization of three scaling factors, labeled as $\boldsymbol{x}_1^0$, $\boldsymbol{x}_2^0$, and $\boldsymbol{x}_3^0$, drawing a triangle region in $s_1 - s_2$ plane. For each iteration, we generate multiple candidates according to the vertex of the current triangle. In Fig. 2(b), each candidate is the midpoint of its line segment (x_m , x_m , x_{cc}, etc.). After determining the candidates, we keep one vertex from the current three vertices for the next iteration, which shows the minimum loss for the given dataset in current iteration. The other two optimal vertices are selected from the remaining vertices and candidates. After several iterations, we choose

the best vertex as the scaling factor. As a result, the average condition number gradually decreases during iterations as Fig. 2 shown. Note that, to ensure quantum speedup, the scaling factor is pre-obtained by running the algorithm on the target dataset.

V. Min-Max Eigenvalue Model

A. Parameterization Using Min-Max Eigenvalue

As mentioned in Section II-B, the QPE module, taking up most of the qubits, is used to extract eigenvalues of the input matrix. Furthermore, since eigenvalues play an important role in characterizing the linear system, their preciseness significantly affects the final output accuracy of the HHL circuit. Thus, the prior knowledge of eigenvalues can naturally improve the output accuracy. However, profiling all eigenvalues requires a huge computational cost that destroys the quantum advantage. Considering that the eigenvalues are inherently related to the hidden information of the portfolio optimization problem, we predict $|\lambda|_{\min}$ and $|\lambda|_{\max}$ and leverage them to set hyperparameters. We choose these two eigenvalues rather than the largest two for two reasons:

- In the view of economics, $|\lambda|_{\min}$ captures the microeconomic details, which describes macroeconomic patterns of asset changes. While $|\lambda|_{\max}$ governs the long-term behavior of the system.
- The condition number κ, serving as the complexity metric, is calculated by $|\lambda|_{\max}/|\lambda|_{\min}$. Thus, the knowledge of these two eigenvalues helps to guide the qubit need of QPE, and then provides the opportunity to quantitatively evaluate the overall circuit.

The hyperparameter settings require that all eigenvalues of the input matrix are within $(-1, 1)$. To ensure this, we perform a standardization operation on the Lagrange matrix as a preprocessing.

$$M_L^{\text{std}} \equiv \frac{l}{|\lambda|_{\max}} M_L, \quad l \in (0, 1) \tag{11}$$

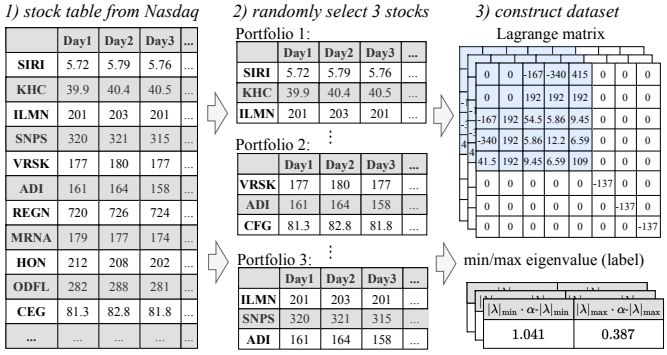
In this paper, we set $l = 0.5$ for simplicity, which makes eigenvalues of M_L^{std} lie in $[1/2\kappa, 1/2)$, but the value of κ does not change.

- **Hamiltonian Simulation Time: $H_t \equiv \pi$.** When $H_t = \pi$, the relationship between the output phase φ of QPE and the eigenvalue λ of M_L^{std} can be represented by a piece-wise function as follows

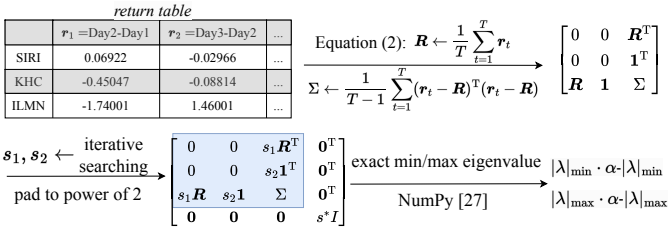
$$\lambda = \begin{cases} 2\varphi, & \varphi < \frac{1}{2}, \\ 2(\varphi - 1), & \varphi > \frac{1}{2}. \end{cases} \tag{12}$$

  Other methods usually set $H_t = 2\pi$ [12], [3] or randomly [20], which makes different value φ may map to the same eigenvalue, leading to low accuracy. In detail, through standardization by $|\lambda|_{\max}$, we ensure $|\lambda| < 1$ so that we can split the interval of $\varphi \in [0, 1)$ into two halves and map to the case of either the eigenvalues being positive or negative bounded by $1/2$. In a word, this one-to-one map results in more preciseness of all eigenvalues.
- **Number of Qubits for QPE: $n_{\text{QPE}} \geq 2 + \left\lceil \log_2 \frac{|\lambda|}{|\lambda| \cdot \epsilon} \right\rceil$.** The minimum n_{QPE} ensures that the absolute error of the solution output by HHL algorithm does not exceed ϵ. Essentially, this calculation uses the quotient of $|\lambda|_{\min}$ and $|\lambda|_{\max}$, which is the condition number. This also further illustrates the fact that the larger the condition number, the more difficult to solve portfolio optimization problems, because we need more qubits to perform QPE.
- **Normalization Constant: $Const \equiv |\lambda|_{\min}$.** The state before final measurement is

$$\sum_{j=1}^{n} \beta_j |\mu_j\rangle_{Q_v} \left(\sqrt{1 - \frac{Const^2}{\lambda_j^2}} |0\rangle_{Q_a} + \frac{Const}{\lambda_j} |1\rangle_{Q_a} \right) \tag{13}$$



(a) Constructing the dataset for 3-assets problem using Nasdaq maket data.The values in the Lagrange matrix need to multiply 10^{-5}.



(b) Details of calculating the dataset.

Fig. 3. Workflow of dataset construction.

For auxiliary qubit in Q_a, $Const \leq |\lambda|_{\min}$ must be satisfied. On the other hand, it is expected that the probability of measuring the auxiliary qubit is close to 1 as high as possible, which means the algorithm has a high success rate. Thus, we set $Const \equiv |\lambda|_{\min}$.

B. Min-Max Eigenvalue Predictor

Most conventional approaches deliver inefficient eigenvalue information. For example, [21] only predicts eigenvalues of Σ, and [22] only estimate the eigenvalue counts. On the contrary, [23] aims to figure out all eigenvalues, necessitating massive computational costs. We develop a predictor to quickly predict the min-max eigenvalue by leveraging historical financial information without any prior knowledge. More importantly, this prediction model is dataset-dependent and can be trained offline, which greatly reduces the computational pressure of the classical part.

We apply support vector regression (SVR) as our basic model. Starting from the matrix in Eq. (10), elements of $s_1\boldsymbol{R}$ are concatenated with the upper triangular portion of Σ into a vector and take this vector as the input of the SVR. The output is the predicted $|\lambda|_{\min}$ and $|\lambda|_{\max}$ multiplied by α-$|\lambda|_{\min}$ and α-$|\lambda|_{\max}$, respectively. Here, we introduce the tunable parameter α to mitigate the truncation error of floating-point.

The training data is collected from three representative stock exchanges listed in Table III. Fig. 3(a) illustrates the detailed workflow for constructing the 3-asset dataset from Nasdaq market. First, we collect the data of 3100 stocks within 251 days and randomly select three stocks to formulate a series of portfolio optimization problems. Then, we can derive the Lagrange matrix and label it with the exact min/max eigenvalues. To be specific, we build a return table by calculating the increments on adjacent days as shwon in Fig. 3(b). Using Eq. (2), we can obtain the historical expected return vector and the covariance matrix to construct the Lagrange matrix. Finally, we can compute the min/max eigenvalue using NumPy [24], regarding it as the label for SVR model training. In particular, since the HHL algorithm can only deal with matrices with size of the power of 2, the Lagrange matrix is padded with zeros and identity matrix. For

TABLE II
MODELS FOR PREDICTING MIN/MAX EIGENVALUES.

Model	# Parameter	#Operations	ℓ_2-loss of $\lvert\lambda\rvert_{\min}, \lvert\lambda\rvert_{\max}$
SVR	1640	1.6×10^3	0.033, 0.39
Decision Tree	1969	2×10^2	0.066, 1.66
MLP	2137	2×10^3	0.034, 0.024
LeNet	20411	2×10^4	0.088, 15.8

example, the 3-asset problem leads to a 5×5 Lagrange matrix, which is then padded by putting a scaled identity matrix at right-bottom.

We have also tried other methods like decision tree and deep learning. Table II evaluates four models for eigenvalue prediction. The other three show limited improvement yet are accompanied by a disproportionately high computational complexity. The effectiveness of SVR lies in its generalization performance, which is demonstrated by portfolio optimization based on the stock pre-selection [25]. Since the computational complexity of SVR is independent of the input space dimensionality [26], our method avoids the computational bottleneck caused by large-scale training to some extent and preserves the quantum advantages.

VI. EVALUATION

A. Experiment Setup

Implementation. SAPO is constructed based on Qiskit [12]. The QPE module and uniformly controlled R_Y gate is implemented by *PhaseEstimation* and *UCRYGate* in the framework. The optimization level of transpiler is set to 0. Scaling factor searching is extended from Nelder-Mead module in SciPy library[27]. The min-max eigenvalue predictor is implemented using Scikit-Learn [28]. Table III gives the configurations of SAPO on each dataset.

Comparison. We compare SAPO with three HHL implementations: basic HHL [1], [2], Qiskit HHL [12] and hybrid HHL [3]. Rebentrost et al. [2] images applying basic HHL [1] to the Black's model, but without any specific parameterizations and experiments. Thus, when compared to basic HHL, we assume eigenvalues have been pre-calculated, only comparing the complexity metric. In Qiskit HHL [12], the Hamiltonian simulation time is 2π, and the uniformly controlled R_Y gate is implemented using *ExactReciprocal*. Hybrid HHL [3] involves both classical and quantum computing. Both Qiskit HHL and hybrid HHL require specifying the number of qubits for QPE, which cannot adaptively balance the trade-off between complexity and accuracy. Thus, we only compare the accuracy metric with them, under the same number of qubits for QPE. Due to the huge complexity of simulating the HHL circuit, we consider the portfolio optimization problem ranging from 2 assets to 6 assets.

B. Scalability and Accuracy Improvement

Complexity analysis. Table IV compares the circuit complexity in terms of the number of necessary qubits, circuit depth, the number of CNOT gates, and the total complexity defined by Eq. 7. Overall, SAPO achieves around 34.64%-36.94% complexity reduction with the absolute error ϵ ranging from $1/16$ to $1/4$. In the case of $\epsilon = 1/8$, the detailed resource consumption is listed. SAPO shows a remarkable reduction, especially N^{CNOT} (achieving a maximum reduction of 99.996% for 2-asset portfolio optimization). According to the value of n^{QPE}, such resource-saving results from our scaling technique that shrinks the condition number. The complexity reduction lightly decreases for smaller ϵ, which comes from the logarithmic function when calculating complexity in Eq. (7).

We find that the complexity is significantly higher when handling 3-asset portfolio optimization problems. This may arise from the infinitesimally small or large condition number in several Lagrange matrices. To confirm this, we also give the median value of each metric in the footnote of Table IV, which turns out to be a much more reasonable value, following the trend of the other cases.

TABLE III
CONFIGURATIONS OF SAPO ON EACH DATASET.

Dataset	#Stocks	Scaling factor search			Min-max predictor	
		s_1	s_2	#iter	α-$\lvert\lambda\rvert_{\min}$	α-$\lvert\lambda\rvert_{\max}$
Nasdaq	3100	0.56	1.9e-3	119	3000	100
NYSE	400	0.25	3.2e-4	152	5000	800
AMEX	40	0.28	6.6e-4	125	4000	400

Accuracy analysis. Fig. 4 gives the accuracy improvement compared to hybrid HHL [3] and Qiskit HHL [12], where the accuracy Δ is defined by Eq. (8). Fig. 4(a) sets a fixed number of qubits, which shows that SAPO improves the accuracy by $1.52\times$ on average compared to Qiskit HHL, and $1.46\times$ on average compared to hybrid HHL. For limited available qubits, SAPO reduces the difficulty of solving linear systems and identifies more reasonable parameter settings for the HHL algorithm, thus, being able to solve more complex problems. In the case of the 5-asset portfolio optimization problem, Fig. 4(b) shows that SAPO improves the accuracy by $1.28\times$ on average compared to Qiskit, and $1.39\times$ compared to hybrid HHL. Particularly, the accuracy of SAPO, with a 4-qubit QPE circuit ($\Delta = 0.96$), outperforms all other approaches, even if they involve a 10-qubit QPE circuit. In a word, SAPO makes a better trade-off between accuracy and scalability by leveraging the features of Black's model and historical financial information.

We conduct an ablation study to quantitatively evaluate the advantages of scaling factor and eigenvalue prediction. Here, SAPO without scaling factor means we train the eigenvalue predictor using original Lagrange matrices. For SAPO without eigenvalue predictor, we adopt the same scheme as Qiskit HHL. Fig. 4(a) demonstrates that the scaling factor contributes most, improving the accuracy $1.74\times$-$1.96\times$ under the constraint of 5-qubits QPE module, and improves the accuracy $1.10\times$-$1.96\times$ for 5-asset portfolio optimization. This mainly benefits from our search algorithm. The accuracy of SAPO without the scaling factor is a little bit lower than hybrid HHL and Qiskit HHL. This is because a huge condition number makes it difficult to predict eigenvalues, leading to a less effective or even counterproductive eigenvalue predictor. However, the other two do not make any use of predicted eigenvalues. The gain from the eigenvalue predictor is $1.21\times$-$1.54\times$ and $1.05\times$-$1.37\times$ for fixed qubit number and fixed asset number, respectively. By leveraging prior knowledge, SAPO is able to capture the min-max eigenvalue more precisely, therefore, achieving high accuracy. Fig. 4(b) also suggests that the technique of scaling factor exhibits more improvement for small QPE modules.

C. Evaluation of the Scaling Factor

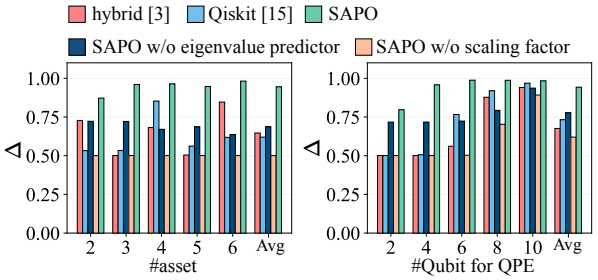
This section provides a detailed evaluation of the scaling factor that serves to squeeze the condition number. Fig. 5(a) draws the distribution of the optimal scaling factor under different datasets, where we randomly select a series of portfolio optimization problems. We can observe that most of their optimal scaling factors lie in a small region, demonstrating that the scaling factor can be static. The figure outlines the region and its edge point of AMEX dataset as an example. SAPO searches the point via the iterative algorithm, which is close to the central in this region.

We also compare the resultant condition number using the scaling factor from the edge point and the optimal scaling factor, as illustrated

TABLE IV
COMPLEXITY COMPARISON WITH BASIC HHL [1], [2].

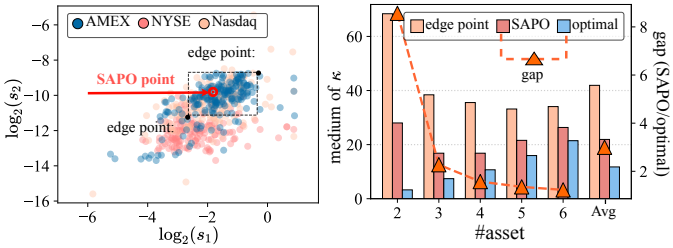
#Asset	The absolute error rate $\epsilon = 1/8$								$\epsilon = 1/16$		$\epsilon = 1/4$	
	N^q		N^d		N^{CNOT}		Θ		Θ		Θ	
	basic HHL	**SAPO**	basic HHL	**SAPO**	basic HHL	**SAPO**	basic HHL	**SAPO**	basic HHL	**SAPO**	basic HHL	**SAPO**
2	19.77	11.80	2.148×10^9	8.016×10^4	1.123×10^9	4.202×10^4	33.82	21.05	35.42	22.65	32.22	19.45
3	21.00	13.87	1.782×10^8	2.523×10^7	9.381×10^7	1.328×10^7	36.76	25.32	38.36	26.93	35.16	23.72
4	21.88	13.23	6.274×10^8	6.454×10^5	3.304×10^8	3.399×10^5	38.17	24.30	39.77	25.91	36.57	22.70
5	22.37	13.21	9.526×10^8	7.062×10^5	5.016×10^8	3.719×10^5	38.94	24.27	40.55	25.87	37.34	22.66
6	22.75	13.42	9.558×10^8	6.737×10^5	5.033×10^8	3.549×10^5	39.56	24.61	41.16	26.22	37.96	23.01
Reduction	39.16%		97.12%		97.12%		36.13%		34.64%		36.94%	

The medium value for #asset=3: $N^q = 13$, $N^d = 4.048 \times 10^5$, $N^{\text{CNOT}} = 2.132 \times 10^5$, $\Theta = 23.94$.



(a) Accuracy under 5-qubit QPE across different number of assets. (b) Accuracy under 5-asset problem across different number of qubits.

Fig. 4. Accuracy comparison with hybrid HHL[3] and Qiskit HHL [12].



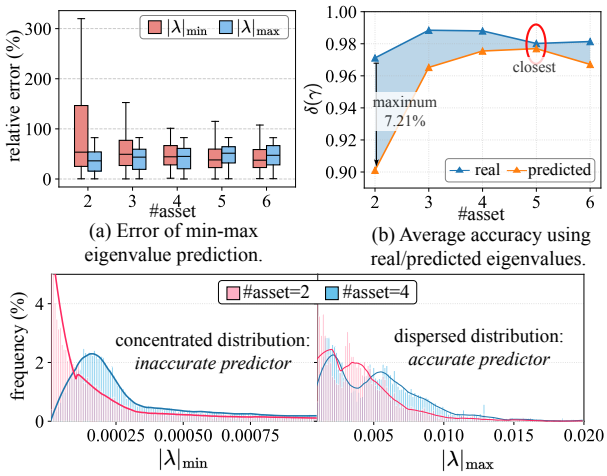
(a) The optimal scaling factor for different portfolios from each dataset. (b) Condition number across different scaling factor.

Fig. 5. Evaluation of the scaling factor.

in Fig. 5(b). Even choosing the edge point still shows a great reduction in the condition number. We also draw the gap between SAPO and the optimal setting, calculated as $\frac{\kappa \text{ of SAPO}}{\kappa \text{ of optimal}}$. The gap is narrowed when more assets are taken into account, from $8.56\times$ (2-asset) to $1.23\times$ (6-asset), demonstrating the validness of static scaling factors. This is because a large number of assets results in more combinations of assets (i.e., more Lagrange matrices), which makes the scaling factor fit the problem better. As a result, the condition number of the Lagrange matrix is dramatically reduced by at least 98.59% through a fixed scaling factor.

D. Evaluation of the Eigenvalue Predictor

In this section, we evaluate the preciseness of the eigenvalue predictor and investigate how it affects the accuracy of the final result. First, we collect the predicted min-max eigenvalues across 1500 cases. Then, we calculate the relative error compared to the real eigenvalues, as shown in the box plot of Fig. 6(a). Overall, the relative error of most predicted values is within 50%. The relative error of some samples is more than 100%, and even exceeds 300% when #asset=2. This is because some eigenvalues are close to zero. Fig. 6(b) depicts the gap of accuracy using the real eigenvalues and the predicted ones, calculated as $\left(1 - \frac{\Delta \text{ using predicted}}{\Delta \text{ using real}}\right) \times 100\%$. The maximum gap is 7.21% under the case of #asset=2, which is still an



(a) Error of min-max eigenvalue prediction. (b) Average accuracy using real/predicted eigenvalues.

(c) Distribution of min/max eigenvalues.

Fig. 6. Evaluation of the min/max eigenvalue predictor.

acceptable accuracy ($\Delta = 0.901$). Especially when #asset=5, the gap almost disappears, resulting from our precise prediction model.

We also picture the distribution of the real $|\lambda|_{\min}$ and $|\lambda|_{\max}$, as shown in Fig. 6(c). We observe that $|\lambda|_{\min}$ is concentrated in the interval $(0, 0.0001]$ when the number of assets is 2, which makes this eigenvalue hard to predict. That's why the relative error of $|\lambda|_{\max}$ is much lower than $|\lambda|_{\min}$ in Fig. 6(a).

VII. CONCLUSION

We propose SAPO to improve the scalability and accuracy for quantum portfolio optimization, indicating the idea that the algorithm design for quantum finance should be based on the deep integration of quantum computing and financial knowledge. SAPO is a novel design of HHL algorithm, which incorporates mean-variance theory and leveraging financial data characteristics. Depending on the equivalent transform, the condition number of the Lagrange matrix is extremely reduced, which improves the scalability. Considering the close connection between eigenvalues and the HHL algorithm, we propose a min-max eigenvalue model to guide parameter settings for highly accurate computation. Our experiments demonstrate that SAPO can achieve lower complexity and higher accuracy than conventional quantum methods.

ACKNOWLEDGEMENT

This work was supported by National Key Research and Development Program of China (No. 2023YFF0905200) and Zhejiang Provincial Natural Science Foundation of China under Grant (No.LR25F020002).

REFERENCES

[1] A. W. Harrow, A. Hassidim, and S. Lloyd, "Quantum algorithm for linear systems of equations," *Physical review letters*, vol. 103, no. 15, p. 150502, 2009.

[2] P. Rebentrost and S. Lloyd, "Quantum computational finance: quantum algorithm for portfolio optimization," *arXiv preprint arXiv:1811.03975*, 2018.

[3] Y. Lee, J. Joo, and S. Lee, "Hybrid quantum linear equation algorithm and its experimental test on ibm quantum experience," *Scientific reports*, vol. 9, no. 1, p. 4778, 2019.

[4] A. Sedighi and D. Jacobson, "Computational challenges and opportunities in financial services," in *Smart Computing and Communication: 4th International Conference, SmartCom 2019, Birmingham, UK, October 11–13, 2019, Proceedings 4*. Springer, 2019, pp. 310–319.

[5] H. Markowitz, "Portfolio selection*," *The Journal of Finance*, vol. 7, no. 1, pp. 77–91, 1952.

[6] H. M. Markowitz and G. P. Todd, *Mean-variance analysis in portfolio choice and capital markets*. John Wiley & Sons, 2000, vol. 66.

[7] D. Qu, E. Matwiejew, K. Wang, J. B. Wang, and P. Xue, "Experimental implementation of quantum-walk-based portfolio optimisation," *Quantum Science and Technology*, 2024.

[8] D. Venturelli and A. Kondratyev, "Reverse quantum annealing approach to portfolio optimization problems," *Quantum Machine Intelligence*, vol. 1, pp. 17 – 30, 2018.

[9] S. Tan, M. Yu, A. Python, Y. Shang, T. Li, L. Lu, and J. Yin, "Hyqsat: A hybrid approach for 3-sat problems by integrating quantum annealer with cdcl," in *2023 IEEE International Symposium on High-Performance Computer Architecture (HPCA)*. IEEE, 2023, pp. 731–744.

[10] G. S. Ravi, P. Gokhale, Y. Ding, W. Kirby, K. Smith, J. M. Baker, P. J. Love, H. Hoffmann, K. R. Brown, and F. T. Chong, "Cafqa: A classical simulation bootstrap for variational quantum algorithms," in *Proceedings of the 28th ACM International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS), Volume 1*. New York, NY, USA: Association for Computing Machinery, 2022, p. 15–29.

[11] R. Ayanzadeh, N. Alavisamani, P. Das, and M. Qureshi, "Frozenqubits: Boosting fidelity of qaoa by skipping hotspot nodes," in *Proceedings of the 28th ACM International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS), Volume 2*, 2023, pp. 311–324.

[12] A. Javadi-Abhari, M. Treinish, K. Krsulich, C. J. Wood, J. Lishman, J. Gacon, S. Martiel, P. D. Nation, L. S. Bishop, A. W. Cross *et al.*, "Quantum computing with qiskit," *arXiv preprint arXiv:2405.08810*, 2024.

[13] Wikipedia contributors, "Condition number — Wikipedia, the free encyclopedia." [Online]. Available: https://en.wikipedia.org/wiki/Condition_number

[14] G. Q. AI, "Quantum computer datasheet," May 2021. [Online]. Available: https://quantumai.google/hardware/datasheet/weber.pdf

[15] IBM, "Quantumcircuit — ibm quantum document," accessed: March 29, 2024. [Online]. Available: https://docs.quantum.ibm.com/api/qiskit/circuit

[16] Y. Saad, *Iterative methods for sparse linear systems*. SIAM, 2003.

[17] J. R. Shewchuk *et al.*, "An introduction to the conjugate gradient method without the agonizing pain," 1994.

[18] R. D. Braatz and M. Morari, "Minimizing the euclidean condition number," *SIAM Journal on Control and Optimization (SICON)*, vol. 32, no. 6, pp. 1763–1768, 1994.

[19] F. Gao and L. Han, "Implementing the nelder-mead simplex algorithm with adaptive parameters," *Computational Optimization and Applications*, vol. 51, no. 1, pp. 259–277, 2012.

[20] R. Yalovetzky, P. Minssen, D. Herman, and M. Pistoia, "Nisq-hhl: Portfolio optimization for near-term quantum hardware," 2021. [Online]. Available: https://www.jpmorgan.com/content/dam/jpm/cib/complex/content/technology/publications/white-paper-nisq-hhl-portfolio-optimization-for-near-team-quantum-hardward.pdf

[21] X. Mestre, "Improved estimation of eigenvalues and eigenvectors of covariance matrices using their sample estimates," *IEEE Transactions on Information Theory*, vol. 54, no. 11, pp. 5113–5129, 2008.

[22] E. Di Napoli, E. Polizzi, and Y. Saad, "Efficient estimation of eigenvalue counts in an interval," *Numerical Linear Algebra with Applications*, vol. 23, no. 4, pp. 674–692, 2016.

[23] R. D. Somma, "Quantum eigenvalue estimation via time series analysis," *New Journal of Physics*, vol. 21, no. 12, p. 123025, 2019.

[24] C. R. Harris, K. J. Millman, S. J. van der Walt, R. Gommers, P. Virtanen, D. Cournapeau, E. Wieser, J. Taylor, S. Berg, N. J. Smith, R. Kern, M. Picus, S. Hoyer, M. H. van Kerkwijk, M. Brett, A. Haldane, J. F. del Río, M. Wiebe, P. Peterson, P. Gérard-Marchant, K. Sheppard, T. Reddy, W. Weckesser, H. Abbasi, C. Gohlke, and T. E. Oliphant, "Array programming with NumPy," *Nature*, vol. 585, no. 7825, pp. 357–362, Sep. 2020.

[25] N. F. Silva, L. P. de Andrade, W. S. da Silva, M. K. de Melo, and A. O. Tonelli, "Portfolio optimization based on the pre-selection of stocks by the support vector machine model," *Finance Research Letters*, vol. 61, p. 105014, 2024.

[26] M. Awad, R. Khanna, M. Awad, and R. Khanna, "Support vector regression," *Efficient learning machines: Theories, concepts, and applications for engineers and system designers*, pp. 67–80, 2015.

[27] P. Virtanen, R. Gommers, T. E. Oliphant, M. Haberland, T. Reddy, D. Cournapeau, E. Burovski, P. Peterson, W. Weckesser, J. Bright, S. J. van der Walt, M. Brett, J. Wilson, K. J. Millman, N. Mayorov, A. R. J. Nelson, E. Jones, R. Kern, E. Larson, C. J. Carey, İ. Polat, Y. Feng, E. W. Moore, J. VanderPlas, D. Laxalde, J. Perktold, R. Cimrman, I. Henriksen, E. A. Quintero, C. R. Harris, A. M. Archibald, A. H. Ribeiro, F. Pedregosa, P. van Mulbregt, and SciPy 1.0 Contributors, "SciPy 1.0: Fundamental Algorithms for Scientific Computing in Python," *Nature Methods*, vol. 17, pp. 261–272, 2020.

[28] F. Pedregosa, G. Varoquaux, A. Gramfort, V. Michel, B. Thirion, O. Grisel, M. Blondel, P. Prettenhofer, R. Weiss, V. Dubourg, J. Vanderplas, A. Passos, D. Cournapeau, M. Brucher, M. Perrot, and E. Duchesnay, "Scikit-learn: Machine learning in Python," *Journal of Machine Learning Research*, vol. 12, pp. 2825–2830, 2011.